Ascon

Implementations

Ascon was designed to be easy to implement, without dependencies on other ciphers, finite field arithmetics or similar. The core permutation can be implemented efficiently in both hardware and software. Find out more on advantages of the Ascon design for optimized software and hardware implementations in the submission document.

Several software and hardware implementations are collected in our GitHub repository. Note that some third-party implementations may implement the submitted version rather than the NIST draft standard version.

Software Reference Implementations

C [git] [zip]:

The repository features both the reference implementation and optimized implementations (64-bit) of all Ascon family members. For a detailed overview of the performance of Ascon on different CPUs, we refer to eBAEAD.

Performance of Ascon C implementations (cycles per byte, for long messages, estimated)

	Ascon-AEAD128	Ascon-Hash256, Ascon-XOF128, Ascon-CXOF128
AMD EPYC 7742	4.2 c/B	12.4 c/B
AMD Ryzen 9 5950X	5.2 c/B	15.8 c/B
Apple M1 (ARMv8)	6.3 c/B	18.5 c/B

	Ascon-AEAD128	Ascon-Hash256, Ascon-XOF128, Ascon-CXOF128
Cortex-A72 (ARMv8)	7.0 c/B	20.0 c/B
Intel Xeon E5-2609 v4	7.2 c/B	21.2 c/B
Intel Core i5-6300U	7.8 c/B	23.1 c/B
Intel Core i5-4200U	10.6 c/B	30.7 c/B
Cortex-A9 (ARMv7)	24.0 c/B	53.9 c/B
Cortex-A7 (NEON)	30.7 c/B	
Cortex-A7 (ARMv7)	41.2 c/B	
ARM1176JZF-S (ARMv6)	42.9 c/B	92.2 c/B

Python [git] [py]:

Reference implementation of all AEAD and hash family members.
Note: The pypi package is not maintained by us.

Software

A collection of third-party software implementations using a variety of languages and platforms is maintained on https://github.com/ascon/ascon_collection.

Hardware

NIST LWC Hardware API reference implementation in VHDL by Robert Primas [git]:

Reference hardware implementations of all AEAD and hash family members by Robert Primas using the NIST LWC Hardware API v1.2.

Simplified Hardware API implementation in Verilog by Robert Primas [git]:

Hardware implementations of the primary variants Ascon-128 and Ascon-Hash by Robert Primas using a simplified interface to reduce the size of the code base.

CAESAR Hardware API reference implementations in VHDL by Hannes Groß [git]:

Reference hardware implementations of Ascon-128 and Ascon-128a by Hannes Groß using the CAESAR Hardware API v1.0. Note that the CAESAR API implies a certain overhead, in particular for lightweight designs like Ascon.

Ascon-128 (CAESAR Hardware API)

Design	Area	Throughput
1 round	9420 GE	4888 Mbps
2 rounds	12989 GE	8482 Mbps
3 rounds	16589 GE	10343 Mbps
6 rounds	27280 GE	12261 Mbps

Additional: Pre-Processor 869 GE, Post-Processor 1032 GE, HDR Buffer 836 GE

Ascon-128a (CAESAR Hardware API)

Design	Area	Throughput
1 round	9680 GE	7326 Mbps
2 rounds	13249 GE	11743 Mbps
4 rounds	20380 GE	16675 Mbps

Additional: Pre-Processor 1491 GE, Post-Processor 1344 GE, HDR Buffer 836 GE

CAESAR Hardware API implementation in VHDL by the Athena project [web]:

Hardware implementation of Ascon-128 and Ascon-128a, including a database of FPGA results for comparison with other CAESAR candidates.

Protected hardware implementation in VHDL by Hannes Groß [git]:

Side-channel protected hardware implementations of Ascon-128 and Ascon-128a by Hannes Groß using domain-oriented masking.

Energy-efficient implementation in VHDL by Michael Fivez [git]:

Energy-efficient implementations of Ascon-128 and Ascon-128a by Michael Fivez, including a comparison with Joltik and MORUS (master's thesis).

Co-Processor

RISC-V Ascon Accelerator [paper] [git]:

A fast and compact co-processor design for Ascon that can perform AEAD/hashing with a performance of about 2 cycles/byte, or about 4 cycles/byte if protection against fault attacks and power analysis is desired. This co-processor requires only 4.7 kGE, or about half the area of dedicated co-processor designs, and is easy to integrate into low-end embedded devices like 32-bit ARM Cortex-M or RISC-V microprocessors.

Runtime (cycles per byte) and code size comparison of Ascon, with/without 1-round Ascon-p hardware acceleration (Co-Proc.) on the RISC-V RI5CY core.

Design	Implementation	Runtime			Codesize
		64 B	1536 B	long	
Ascon-128 (-O3)	SW	164.3 c/B	110.6 c/B	108.3 c/B	11716 B
Ascon-Hash (-O3)	SW	306.9 c/B	208.0 c/B	203.8 c/B	20244 B
Ascon128	SW+Coproc.	4.2 c/B	2.2 c/B	2.1 c/B	888 B
Ascon-Hash	SW+Coproc.	4.6 c/B	2.6 c/B	2.5 c/B	484 B

Benchmarks

Athena project's CAESAR Hardware API benchmarks for FPGA and ASIC [web]:

Benchmarks and tools for hardware implementations. See "Publications" for various related publications.

eBACS/SUPERCOP: ECRYPT Benchmarking of Cryptographic Systems [web]:

Benchmark of software implementations of LWC and CAESAR candidates and other AEAD designs on a wide range of platforms.

Rhys Weatherley's microcontroller benchmarks (ARM and AVR) [web] [git]:

Benchmark of software implementations of LWC candidates on 8-bit and 32-bit platforms: ARM Cortex M3, ESP32 Arduino, and ATmega2560.

LaS3's LWC microcontroller benchmarks [web] [git] [talk]:

Benchmark of software implementations of LWC candidates on microcontrollers: Arduino Uno R3, STM32F1 "bluepill", Espressif ESP32 WROOM, STM32 NUCLEO-

F746ZG, Sipeed Maixduino RISC-V 64.

FELICS-AE benchmarks [git]:

Benchmark of LWC candidates based on the FELICS framework.

Ankele & Ankele's software benchmarks for CAESAR [git] [paper]:

Software benchmarking of 2nd round CAESAR candidates